

AI Powered Debt Relief's Financial Recovery Platform

Introduction:

Financial debt has become one of the most significant economic challenges faced by individuals and families. Managing multiple loans, credit card bills, and repayment schedules often becomes difficult without proper financial guidance. Most existing financial advisory platforms provide only numerical calculations and lack intelligent decision-support mechanisms capable of recommending personalized repayment strategies.

The AI Debt Relief project is an intelligent financial advisory system designed to help users analyze their debt situations and receive AI-powered recommendations for debt settlement and repayment planning. The application combines financial analysis, settlement prediction, and negotiation strategy generation into a single web-based platform.

The project follows a modern full-stack architecture, where the frontend is developed using React.js and Vite, while the backend uses FastAPI to expose RESTful APIs. Specialized backend modules perform financial analysis, settlement prediction, and recommendation generation. The modular architecture ensures scalability, maintainability, and ease of future enhancements.

Project Description:

AI Debt Relief is an intelligent web application that assists users in evaluating their financial health and selecting suitable debt repayment strategies.

The application collects financial information such as:

- Monthly Income
- Monthly Expenses
- Total Outstanding Debt
- Loan Details
- Interest Rates
- Repayment Capacity

Using AI-driven financial processing modules, the system analyzes this information and provides:

- Debt settlement estimation
- Personalized repayment recommendations
- Negotiation strategy suggestions
- Financial health insights
- Intelligent fallback recommendations

Unlike traditional financial calculators, the platform integrates multiple financial processing components through FastAPI services, enabling real-time decision support.

Project WorkFlow:

Epic 1. FinRelief AI Application Development s System Setup

Story 1: Python Backend Environment Setup

Story 2: Backend Dependency Installation Using Requirements.txt Story 3:

Frontend Setup (React.js + Vite)

Story 4: Project Structure C Full-Stack Directory Organization

Epic 2. AI Integration s Financial Processing Setup

Story 1: FastAPI Backend C API Endpoints

Story 2: Financial Engine Module

Story 3: Settlement Prediction System

Story 4: AI Negotiation Strategy Engine

Story 5: Fallback Logic Implementation

Epic 3. Database Management s Financial Data Storage Setup

Story 1: API Development Functionality

Story 2: Loan C Settlement Processing Functionality Story 3:

Data Handling

Epic 4. Frontend Integration s User Interface Development

Story 1: User Interface Development

Story 2: Frontend Communication With FastAPI

Story 3: Financial Health Metrics C Settlement Data Visualization Story 4:

UI Enhancements

Epic 5. Testing, Debugging s Performance Optimization

Story 1: System Testing

Story 2: Backend Error Handling C AI Fallback Management

Story 3: Performance Optimization C Secure Session Management

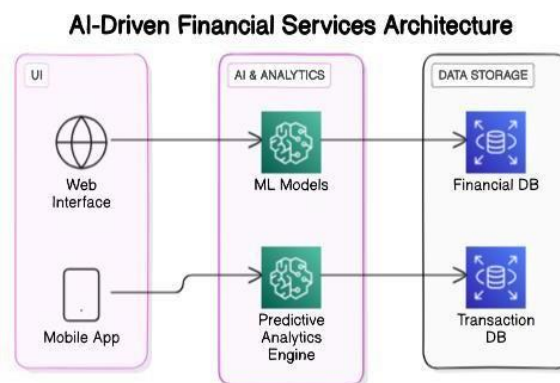
Epic 6. Version Control, Project Finalization s Deployment Readiness

Story 1: GitHub Repository Setup C Version Control Management

Story 2: Project Cleanup C Modular Folder Structure Organization Story

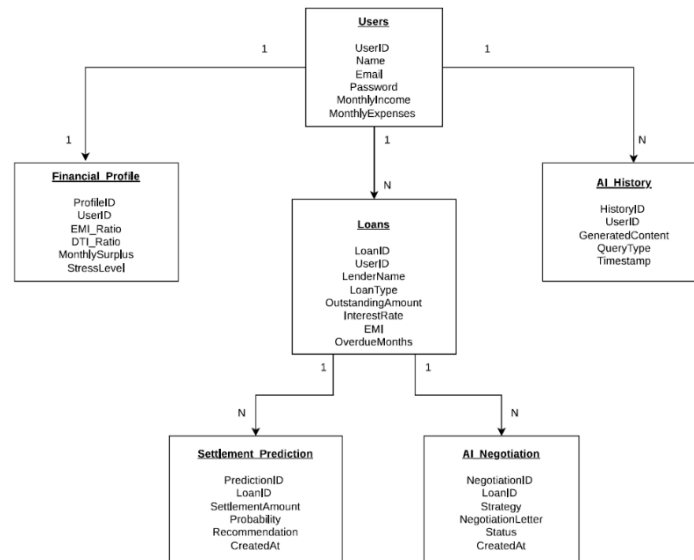
3: Deployment Configuration C Production Readiness Setup

Technical architecture:



ER Diagram:

The ER Diagram of the AI Financial Advisor (AIFA) illustrates the relationships between users, financial profiles, goals, analyses, AI recommendations, and chat history, ensuring efficient data management and personalized financial planning.



Pre-requisites:

1. Python 3.11+
2. FastAPI
3. React.js
4. Vite
5. Google Gemini API
6. SQLAlchemy ORM
7. SQLite
8. Git Version Control
9. GitHub Repository Hosting
10. Python Virtual Environment (venv)
11. Visual Studio Code
12. JWT Authentication

Epic 1 – Project Initialization:

Story 1 – Project Setup:

Description:

Initialized the FinRelief AI project using React.js and Vite. Created the basic folder structure and configured the development environment.

Features

- React + Vite Project
- Folder Structure
- Package Installation
- Git Repository
- Development Environment Setup

```
(.venv) PS C:\AI_Project> python --version
Python 3.13.14
(.venv) PS C:\AI_Project> python -m venv venv
(.venv) PS C:\AI_Project> venv\Scripts\activate
(venv) PS C:\AI_Project> 
```

Story 2 – Backend Setup:

Description:

Configured FastAPI backend for API development.

Features

- FastAPI Installation
- Python Virtual Environment
- API Structure
- Backend Server

```
(venv) PS C:\AI_Project> pip install -r requirements.txt
Collecting annotated-types==0.7.0 (from -r requirements.txt (line 1))
  Using cached annotated_types-0.7.0-py3-none-any.whl.metadata (15 kB)
Collecting anyio==4.14.1 (from -r requirements.txt (line 2))
  Using cached anyio-4.14.1-py3-none-any.whl.metadata (4.6 kB)
Collecting bcrypt<5.0 (from -r requirements.txt (line 3))
  Downloading bcrypt-4.3.0-cp39-abi3-win_amd64.whl.metadata (10 kB)
Collecting certifi==2026.6.17 (from -r requirements.txt (line 4))
  Using cached certifi-2026.6.17-py3-none-any.whl.metadata (2.5 kB)
Collecting cffi==2.0.0 (from -r requirements.txt (line 5))
  Using cached cffi-2.0.0-cp313-cp313-win_amd64.whl.metadata (2.6 kB)
Collecting charset-normalizer==3.4.7 (from -r requirements.txt (line 6))
  Using cached charset_normalizer-3.4.7-cp313-cp313-win_amd64.whl.metadata (41 kB)
Collecting click==8.4.2 (from -r requirements.txt (line 7))
  Using cached click-8.4.2-py3-none-any.whl.metadata (2.6 kB)
Collecting colorama==0.4.6 (from -r requirements.txt (line 8))
  Using cached colorama-0.4.6-py2.py3-none-any.whl.metadata (17 kB)
Collecting cryptography==49.0.0 (from -r requirements.txt (line 9))
  Using cached cryptography-49.0.0-cp311-abi3-win_amd64.whl.metadata (4.3 kB)
Collecting dnspython==2.8.0 (from -r requirements.txt (line 10))
  Using cached dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
Collecting ecdsa==0.19.2 (from -r requirements.txt (line 11))
  Using cached ecdsa-0.19.2-py2.py3-none-any.whl.metadata (29 kB)
Collecting email_validator==2.2.0 (from -r requirements.txt (line 12))
  Using cached email_validator-2.2.0-py3-none-any.whl.metadata (25 kB)
Collecting fastapi==0.115.0 (from -r requirements.txt (line 13))
  Using cached fastapi-0.115.0-py3-none-any.whl.metadata (27 kB)
Collecting h11==0.16.0 (from -r requirements.txt (line 14))
  Using cached h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Collecting httptools==0.8.0 (from -r requirements.txt (line 15))
  Using cached httptools-0.8.0-cp313-cp313-win_amd64.whl.metadata (3.7 kB)
Collecting idna==3.18 (from -r requirements.txt (line 16))
```

Story 3 – Version Control:

Description:

Configured GitHub repository and project version control.

Features

- Git Initialization
- GitHub Repository
- Initial Commit
- Branch Management

```
(.venv) PS C:\AI_Project\frontend> npm install

up to date, audited 138 packages in 5s

21 packages are looking for funding
  run `npm fund` for details

2 vulnerabilities (1 moderate, 1 high)

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
npm warn allow-scripts 1 package has install scripts not yet covered by allowScripts:
npm warn allow-scripts   esbuild@0.21.5 (install: (install scripts present))
npm warn allow-scripts
npm warn allow-scripts Run `npm approve-scripts --allow-scripts-pending` to review, or `npm approve-scripts <pkg>` to allow.
(.venv) PS C:\AI_Project\frontend> npm run dev

> finrelief-ai-frontend@1.0.0 dev
> vite

VITE v5.4.21 ready in 743 ms

→ Local:   http://localhost:5173/AI_DEBT/
→ Network: http://10.213.171.8:5173/AI_DEBT/
→ press h + enter to show help
```

Story 4– Project Structure Setup:

Description:

Project Structure Setup The project follows a modular full-stack architecture with separate modules for frontend pages, backend APIs, authentication, financial engine, settlement engine, and AI negotiation strategy generation.

```
(.venv) PS C:\AI_Project> dir

Directory: C:\AI_Project

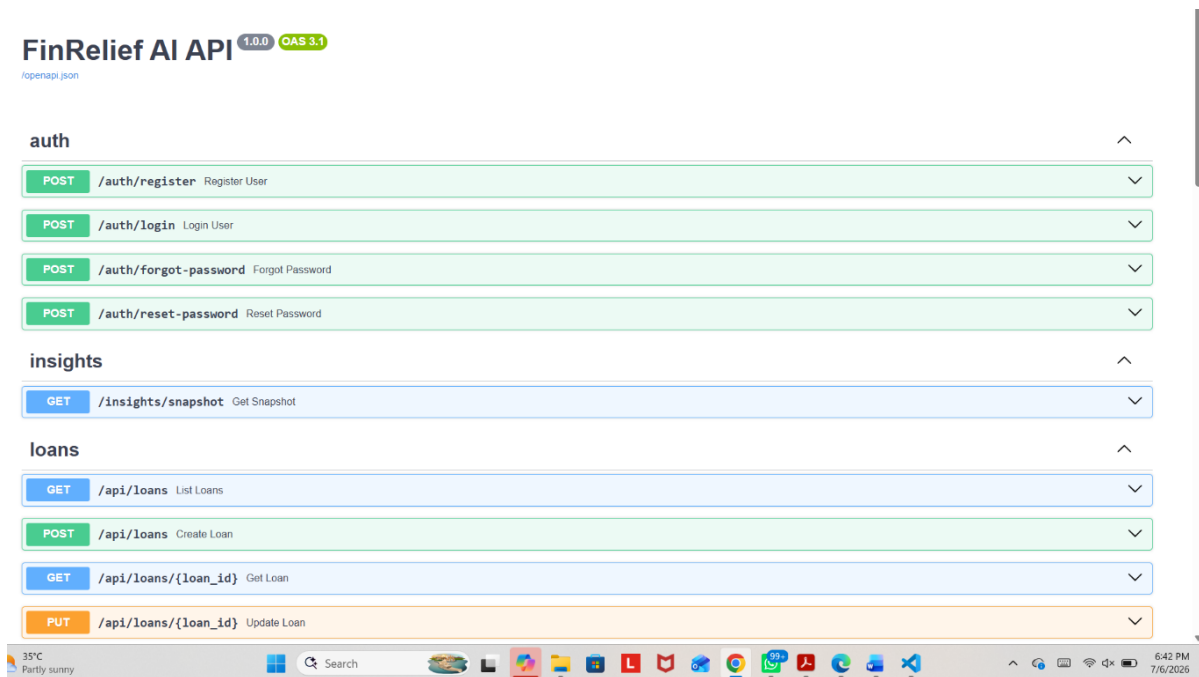
Mode                LastWriteTime         Length Name
----                -
d-----          7/5/2026   1:15 AM             .github
d-----          7/4/2026  10:25 PM             .pytest_cache
d-----          7/4/2026   9:38 PM             .venv
d-----          7/6/2026   4:27 PM             backend
d-----          7/6/2026   6:31 PM             frontend
d-----          7/6/2026   6:25 PM             venv
-a-----          7/5/2026   4:40 PM             156 .env.example
-a-----          7/4/2026  10:29 PM             508 .gitignore
-a-----          7/5/2026  12:49 AM             82336 Design and Analysis of ER Diagram for FinRelief AI.pdf
-a-----          7/6/2026   6:30 PM             284 package-lock.json
-a-----          7/4/2026  10:30 PM             416 package.json
-a-----          7/5/2026  12:49 AM             4295 README.md
-a-----          7/5/2026   4:00 PM             328 render.yaml
-a-----          7/5/2026   4:25 PM             1386 requirements.txt
-a-----          7/5/2026   3:56 PM             15 runtime.txt
```

Epic 2: AI Integration s Financial Processing Setup:

Story 1 – FastAPI Backend s API Endpoints:

Description:

FastAPI Backend C API Endpoints The system implements a clean REST API using FastAPI with multiple endpoints handling user authentication, loan management, financial health calculation, settlement prediction, and AI negotiation strategy generation.



Story 2 – Financial Engine Module:

Description:

Financial Engine Module The platform calculates real-time financial health metrics including EMI-to-income ratio, debt-to-income ratio, monthly surplus, total outstanding debt, and financial stress level based on the borrower's income, expenses, and loan portfolio.

Function: EMI C Debt-to-Income Ratio Calculation

Calculates EMI ratio and debt-to-income percentage using borrower monthly income, expenses, and total loan obligations for financial health analysis.

```
PS C:\AI_Projct> cd backend
PS C:\AI_Projct\backend> python financial_engine.py
{'Total EMI': 18000, 'Outstanding Amount': 700000, 'Monthly Surplus': 17000, 'EMI Ratio (%)': 30.0, 'Debt To Income (%)': 116.6, 'Risk Category': 'Medium'}
```

Story 3 – Settlement Prediction System:

Description:

The system calculates optimal settlement percentages and risk categories for each loan by analyzing outstanding amount, overdue months, loan type, and borrower financial profile to provide data-driven settlement recommendations.

```
PS C:\AI_Projct\backend> python settlement_prediction.py
Settlement Prediction Report
-----
{'Loan ID': 1, 'Lender': 'SBI', 'Settlement %': 55, 'Risk Score': 15, 'Risk Category': 'Low'}
{'Loan ID': 2, 'Lender': 'HDFC', 'Settlement %': 65, 'Risk Score': 45, 'Risk Category': 'High'}
```

Story 4 - AI Negotiation Strategy Engine

Description:

An AI-powered negotiation strategy system is implemented to:

- Analyze borrower's complete financial profile and loan data
- Generate personalized settlement strategies using Google Gemini API
- Produce lender-specific professional negotiation letters
- Activate rule-based fallback engine when API key is unavailable

```
PS C:\AI_Projct\backend> python ai_negotiation.py
Gemini API Key not found.
Using Rule-Based Strategy...

Negotiation Strategy

• Explain your financial hardship.
• Request a lower settlement amount.
• Ask for EMI restructuring if settlement is not possible.
• Maintain polite and professional communication.
```

Story 5 - Fallback Logic Implementation:

Description:

Rule-based fallback mechanisms and error handling modules are implemented to ensure system stability and uninterrupted AI functionality without an active Gemini API key.

```
PS C:\AI_Projct\backend> python ai_negotiation.py
Gemini API Key not found.
Using Rule-Based Strategy...

Negotiation Strategy

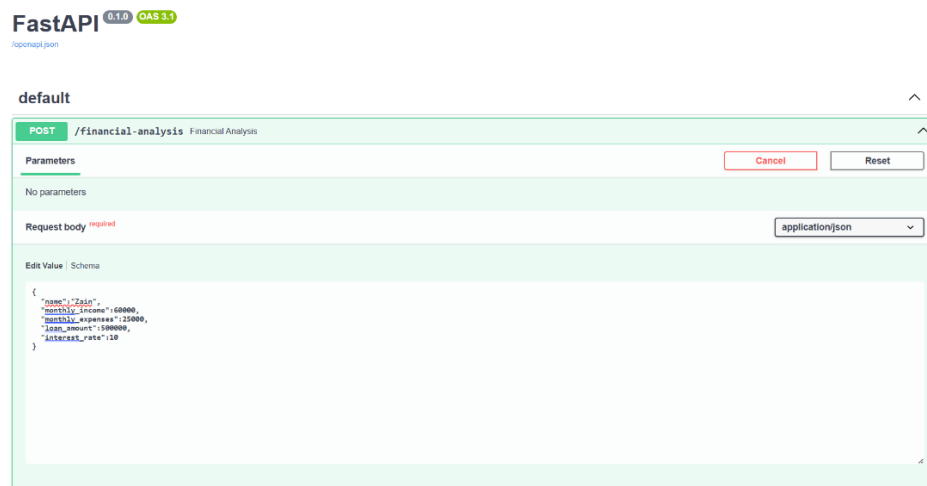
• Explain your financial hardship.
• Request a lower settlement amount.
• Ask for EMI restructuring if settlement is not possible.
• Maintain polite and professional communication.
```

Epic 3: Database Management s Financial Data Storage Setup:

Story 1 -API Development Functionality:

Description:

Accepts frontend requests with loan details and financial profile Validates input using Pydantic schemas
Calls Google Gemini API via ai_engine.py Returns structured JSON responses to frontend



Story 2 - Loans Settlement Processing Functionality:

Description:

Stores and retrieves loan records using SQLAlchemy ORM

Calculates settlement percentage based on overdue months and loan type Assigns

High, Medium, or Low priority to each loan

Generates negotiation letters with lender-specific content

```
PS C:\AI_Projct\backend> python loan_processing.py
Loan Processing Report
-----
Settlement Percentage: 60 %
Priority: Medium

Dear State Bank of India,

I am currently facing financial difficulties and request a settlement for my outstanding loan.

Suggested Settlement: 60%

Thank you for your understanding.

Regards,
Borrower

PS C:\AI_Projct\backend> █
```

Story 3 - AI Powered Debt Relief's Financial Recovery Platform: Description:

Structured loan and user data stored in SQLite database

Efficient query retrieval per user session

JWT token-based authentication for all protected routes

```
(venv) C:\AI-Debt-Relief\backend>python data_handling_demo.py
Stored Users
-----
ID: 1, Email: zain@example.com, Income: 60000.0, Expenses: 25000.0

(venv) C:\AI-Debt-Relief\backend> █
```

Epic 4: Frontend Integration & User Interface Development:

Story 1 - User Interface Development:

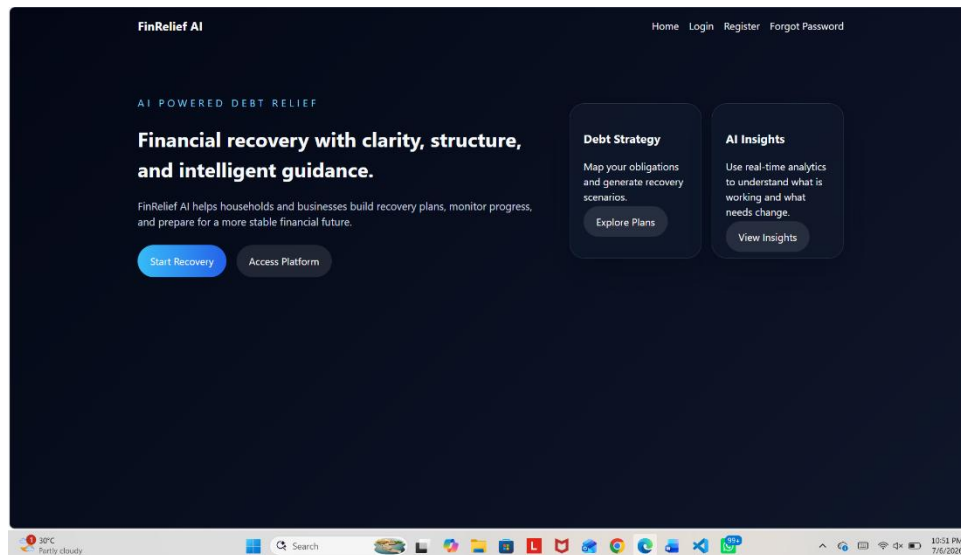
Description:

User Interface Development The frontend is built using React.js and Vite providing a modern dark-themed professional interface.

Users can:

Register and login securely on the Login page

- View financial overview on the Dashboard page
- View settlement predictions on the Settlement Predictor page
- Generate negotiation letters on the Negotiation Email page
- View borrower rights on the Know Your Rights page
- Review past AI outputs on the History page



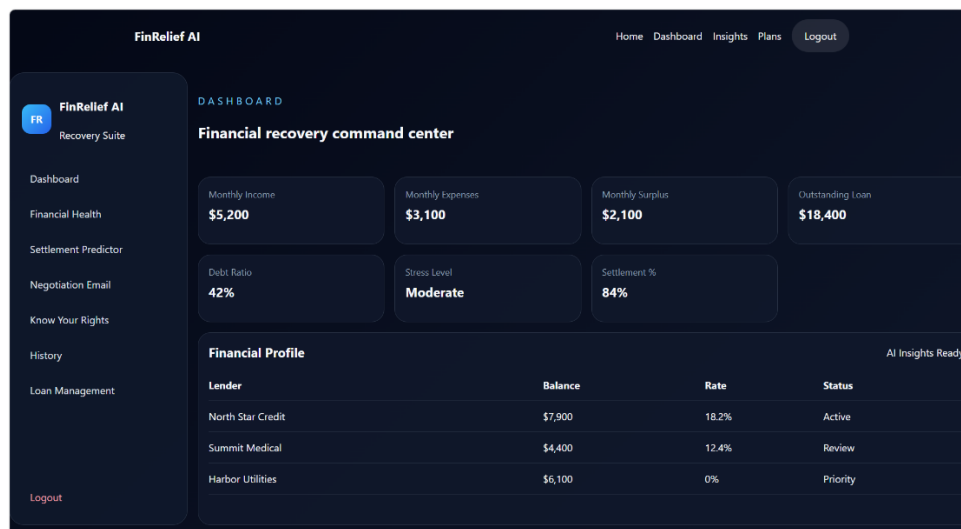
Story 2 - Frontend communicates with FastAPI:

Description:

Backend Integration Frontend communicates with FastAPI backend APIs through Axios with JWT interceptors to process requests and display real-time financial data and AI-generated negotiation content.

Key Functionalities

- Frontend communicates with FastAPI backend APIs using Axios
- JWT interceptors securely attach authentication tokens to API requests
- Real-time financial data is displayed dynamically on the dashboard
- Backend processes borrower loan and financial profile data
- Settlement prediction results are fetched from backend APIs
- AI-generated negotiation strategies are displayed instantly
- Axios interceptors handle unauthorized responses automatically
- Secure authentication flow is maintained across protected routes
- Smooth communication between frontend and backend modules ensured
- Responsive dashboard visualization improves borrower interaction experience



Story 3 - Financial Health Metrics & Settlement Data Visualization:

Description:

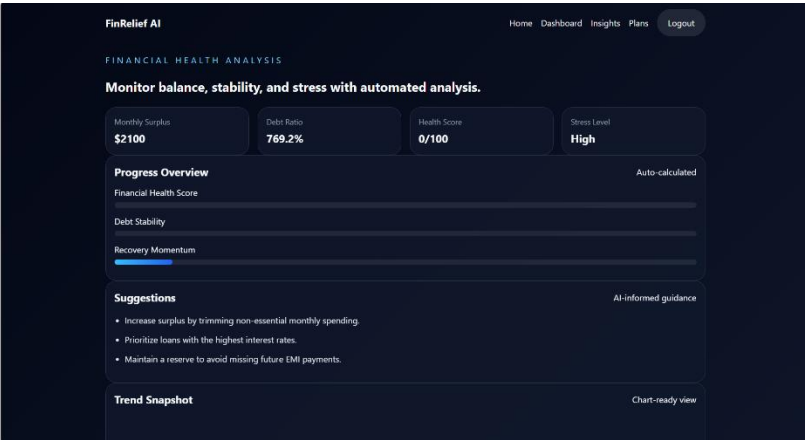
The Financial Data Visualization module provides borrowers with a clear and interactive overview of their financial condition through dynamic dashboard components and real-time analytics. The system visually represents important financial metrics such as monthly surplus, EMI-to-income ratio, debt stress level, settlement percentages, and overall financial health status to help users better understand their debt recovery progress and financial stability.

Key Functionalities

- Monthly surplus metrics displayed dynamically on the dashboard
- EMI-to-income ratio visualization for financial analysis
- Debt stress level badges generated based on borrower financial profile
- Settlement percentage displayed for each loan account
- Financial health progress meters integrated for borrower monitoring
- Real-time financial analytics fetched through backend APIs
- Interactive dashboard cards for financial overview visualization
- Responsive UI components for better user experience
- Color-coded indicators for risk and stress level analysis
- Smooth frontend rendering of borrower financial insights

Financial Dashboard Interface

Displays borrower financial overview including active loans, monthly surplus, debt stress analysis, and real-time financial health metrics through an interactive dashboard.



Story 4 - UI Enhancements:

Description:

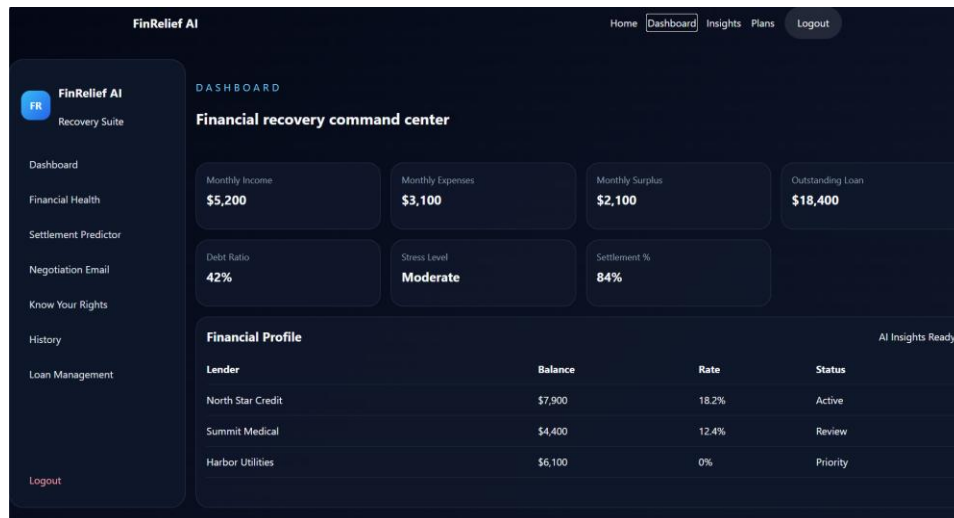
The Financial Data Visualization module provides borrowers with a clear and interactive overview of their financial condition through dynamic dashboard components and real-time analytics. The system visually represents important financial metrics such as monthly surplus, EMI-to-income ratio, debt stress level, settlement percentages, and overall financial health status to help users better understand their debt recovery progress and financial stability.

Key Functionalities

- Monthly surplus metrics displayed dynamically on the dashboard
- EMI-to-income ratio visualization for financial analysis
- Debt stress level badges generated based on borrower financial profile
- Settlement percentage displayed for each loan account
- Financial health progress meters integrated for borrower monitoring
- Real-time financial analytics fetched through backend APIs
- Interactive dashboard cards for financial overview visualization
- Responsive UI components for better user experience
- Color-coded indicators for risk and stress level analysis
- Smooth frontend rendering of borrower financial insights

Financial Dashboard Interface

Displays borrower financial overview including active loans, monthly surplus, debt stress analysis, and real-time financial health metrics through an interactive dashboard.



Epic 5: Testing, Debugging & Performance Optimization:

Story 1 - System Testing:

Description:

The system testing phase focuses on validating the stability, accuracy, and reliability of the FinRelief AI platform across multiple borrower financial scenarios and loan management workflows. Different loan types, financial profiles, settlement prediction cases, and AI-generated negotiation processes are tested to ensure smooth platform functionality and accurate financial analysis under real-world conditions.

Key Functionalities

- Multiple loan types tested for financial processing accuracy

- Different borrower financial profile scenarios validated
- Settlement prediction calculations verified across various debt conditions
- Missing Google Gemini API key scenarios tested successfully
- Invalid user input handling and validation implemented
- Frontend and backend integration testing performed
- API response validation conducted for financial modules
- AI-generated negotiation strategy outputs tested
- Error scenarios and edge cases handled efficiently
- Stable system behavior verified under different financial conditions

FinRelief AI API

OpenAPI JSON

AI Powered Debt Relief Platform

Authentication

POST	/auth/register	Register
POST	/auth/login	Login

Financial Profile

GET	/profile/	Get Profile
PUT	/profile/update	Update Profile

Loans

GET	/loans/	Get Loans
GET	/loans/{loan_id}	Get Loan
POST	/loans/add	Add Loan
PUT	/loans/update/{loan_id}	Update Loan
DELETE	/loans/delete/{loan_id}	Delete Loan

AI Services

GET	/ai/financial-health	Financial Health
GET	/ai/settlement-predictor	Settlement Prediction
GET	/ai/negotiation-email	Negotiation Email
GET	/ai/borrower-rights	Borrower Rights

default

GET	/	Home
GET	/health	Health
GET	/dashboard	Dashboard
GET	/settlement	Settlement
GET	/rights	Rights

Schemas

HTTPValidationError	Expand all	object
ValidationError	Expand all	object

FinRelief AI

HomeDashboardInsightsPlansLogout

FR

FinRelief AI

Recovery Suite

Dashboard

Financial Health

Settlement Predictor

Negotiation Email

Know Your Rights

History

Loan Management

Logout

DASHBOARD

Financial recovery command center

Monthly Income

\$5,200

Monthly Expenses

\$3,100

Monthly Surplus

\$2,100

Outstanding Loan

\$18,400

Debt Ratio

42%

Stress Level

Moderate

Settlement %

84%

Financial Profile

AI Insights Ready

Lender	Balance	Rate	Status
North Star Credit	\$7,900	18.2%	Active
Summit Medical	\$4,400	12.4%	Review
Harbor Utilities	\$6,100	0%	Priority

Story 2- Backend Error Handling & AI Fallback Management

Description:

The Error Handling module is designed to ensure stable, secure, and uninterrupted operation of the FinRelief AI platform during financial processing and AI-powered negotiation workflows. Structured exception handling mechanisms and rule-based fallback systems are implemented across backend modules to manage API failures, invalid user inputs, unexpected server responses, and AI generation issues efficiently.

Key Functionalities

- try-except blocks implemented across backend processing modules
- Rule-based fallback system activated during Gemini API failures
- Stable AI-generated responses ensured without active API access
- Invalid user input handling and validation implemented
- Frontend displays user-friendly error and validation messages
- API timeout and unexpected response handling configured
- Backend exception management prevents application crashes
- Structured error responses returned through FastAPI APIs
- Smooth frontend-backend communication maintained during failures
- Improved platform stability and reliable borrower experience ensured

Story 3 - Performance Optimization & Secure Session Management

Description:

The performance optimization phase focuses on improving the efficiency, stability, authentication handling, and overall responsiveness of the FinRelief AI platform. Backend processing, database operations, authentication workflows, and API communication mechanisms are optimized to ensure smooth financial processing, fast response handling, and reliable borrower interaction across all application modules.

Key Functionalities

- JWT token expiry configured for 120-minute secure session management
- SQLite optimized with `check_same_thread=False` for concurrent request handling
- Axios interceptors implemented to prevent duplicate authentication headers
- Werkzeug password hashing optimized for secure and fast verification
- Backend API response handling improved for better performance
- Database query execution optimized for real-time financial processing
- Authentication workflow enhanced for stable user session management
- Frontend-backend communication latency reduced
- Efficient handling of concurrent borrower requests implemented

Epic 6: Version Control, Project Finalization & Deployment Readiness:

Story 1 - GitHub Repository Setup & Version Control Management:

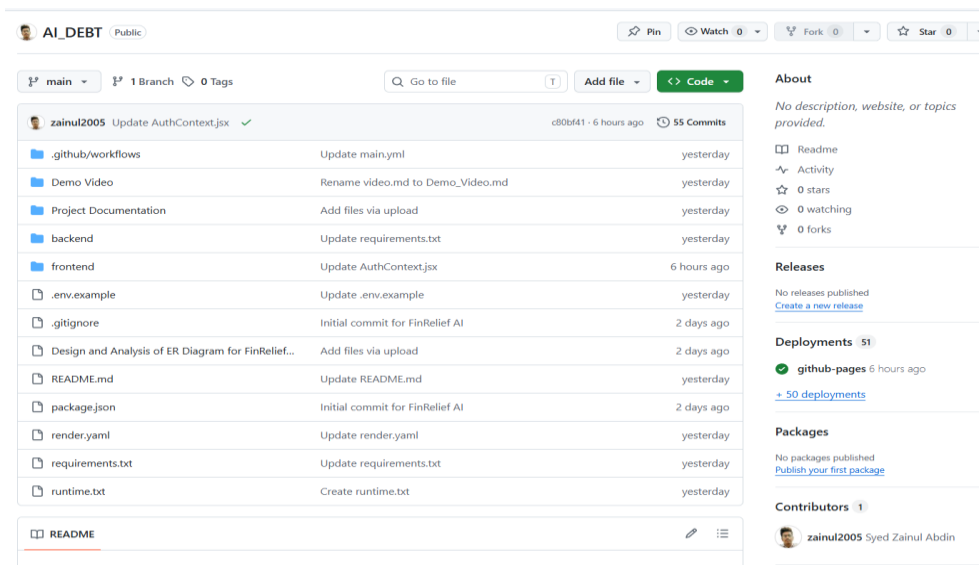
Description:

The GitHub Repository Setup phase focuses on maintaining organized source code management, version tracking, and collaborative development workflows for the FinRelief AI platform. The complete project source code, including frontend, backend, database modules, and AI integration components, is pushed to GitHub to ensure secure repository management and streamlined

development practices.

Key Functionalities

- Project source code pushed successfully to GitHub repository
- Git-based version control workflow implemented
- Backend and frontend modules organized systematically
- Repository structure maintained for scalable development
- Commit history tracked for project modifications and updates
- Secure source code backup and collaboration enabled
- Branch management and code version tracking implemented
- Project files synchronized with GitHub repository
- Development workflow streamlined through Git integration
- Version control maintained for future deployment and enhancements



Story 2 – Project Cleanup & Modular Folder Structure Organization

Description:

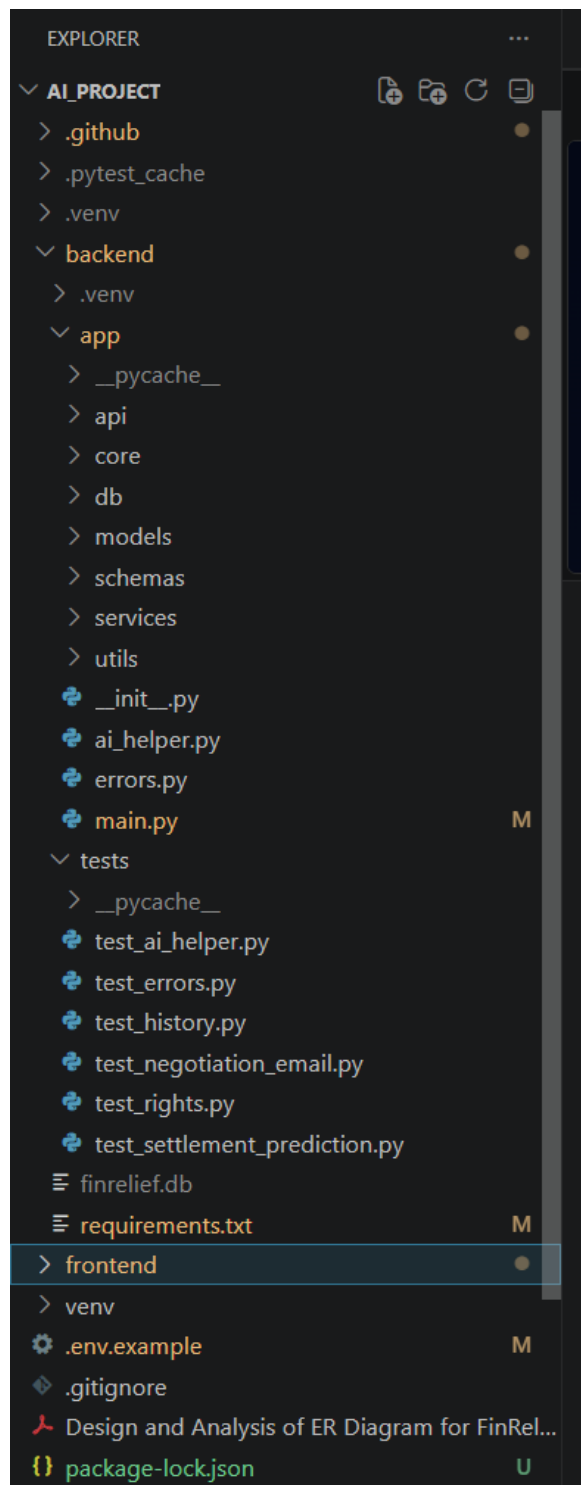
The project cleanup phase focuses on organizing the FinRelief AI application into a clean, scalable, and maintainable modular architecture. Unnecessary temporary files, cache directories, and unused resources are removed to optimize project performance and improve development workflow efficiency.

The application structure is reorganized into separate modules for app configuration, API routes, database models, business services, frontend components, and utility functions to ensure better code readability, maintainability, and deployment readiness.

Key Functionalities

- Unnecessary files and cache directories removed successfully
- Project structure organized into modular folders and components
- Separate app, routes, models, services, and utils modules maintained
- Backend and frontend source code cleaned and optimized
- Improved code readability and maintainability achieved
- Scalable folder architecture implemented for future enhancements
- Duplicate and unused resources removed from the project
- Modular organization improved development workflow efficiency

- Structured source code management prepared for deployment
- Clean and professional project architecture maintained



Story 3 - Deployment Configuration & Production Readiness Setup:

Description:

The deployment readiness phase focuses on preparing the FinRelief AI platform for production

deployment by configuring frontend builds, backend server setup, and environment variable management. The application is optimized to ensure stable execution, scalable deployment, secure API communication, and smooth integration of all financial processing and AI-powered modules in a production environment.

The frontend application is prepared using Vite production build optimization, while backend APIs are configured using the Uvicorn ASGI server for high-performance request handling and scalable FastAPI deployment. Environment variables are securely managed through .env configuration files to maintain safe handling of database credentials, API keys, authentication secrets, and deployment settings.

Key Functionalities

- Frontend prepared for deployment using Vite production build
- Backend APIs configured with Uvicorn ASGI server
- Environment variables successfully configured in .env file
- Production-ready FastAPI backend deployment setup implemented
- Secure handling of API keys and authentication secrets ensured
- Frontend and backend deployment workflow validated
- Optimized production build generated for scalable application hosting
- Deployment configuration organized for cloud/server environments
- API communication and authentication flow verified for production mode
- Application prepared for stable and efficient deployment execution

```
PS C:\AI_Project> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned)
(.venv) PS C:\AI_Project> cd frontend
(.venv) PS C:\AI_Project\frontend> npm run build

> finrelief-ai-frontend@1.0.0 build
> vite build

vite v5.4.21 building for production...
✓ 105 modules transformed.
dist/index.html                                0.52 kB | gzip: 0.33 kB
dist/assets/index-Csmmas7N.css                 5.09 kB | gzip: 1.68 kB
dist/assets/AuthLayout-BQWfMACX.js            0.73 kB | gzip: 0.34 kB
dist/assets/PlansPage-3AAjF_4E.js             0.80 kB | gzip: 0.41 kB
dist/assets/InsightsPage-D7AFCPfn.js          0.82 kB | gzip: 0.44 kB
dist/assets/ResetPasswordPage-XFsbT96b.js     1.12 kB | gzip: 0.66 kB
dist/assets/ForgotPasswordPage-Bqgpamrh.js    1.13 kB | gzip: 0.68 kB
dist/assets/LandingPage-D7xDOjvr.js           1.30 kB | gzip: 0.59 kB
dist/assets/RightsPage-CaY-jbxU.js            1.38 kB | gzip: 0.65 kB
dist/assets/LoginPage-D5Ifz_7X.js             1.65 kB | gzip: 0.83 kB
dist/assets/RegisterPage-D-hl1Ime.js          1.69 kB | gzip: 0.85 kB
dist/assets/HistoryPage-CEZ2nOV7.js           2.27 kB | gzip: 0.88 kB
dist/assets/NegotiationEmailPage-Mlvh8oFM.js  2.96 kB | gzip: 1.26 kB
dist/assets/FinancialHealthPage-DPuHhc-x.js   2.97 kB | gzip: 0.88 kB
dist/assets/SettlementPredictorPage-DVHaH7Ka.js 3.18 kB | gzip: 1.10 kB
dist/assets/DashboardPage-DqRF4OmW.js         3.39 kB | gzip: 0.99 kB
dist/assets/LoanManagementPage-BZFPsZij.js    4.12 kB | gzip: 1.40 kB
dist/assets/index-anE7vZ-4.js                 220.06 kB | gzip: 73.98 kB
✓ built in 1.55s
(.venv) PS C:\AI_Project\frontend>
```

Conclusion:

The FinRelief AI – AI-Powered Debt Relief & Financial Recovery Platform successfully demonstrates the application of modern full-stack web technologies and intelligent AI-powered

negotiation systems in improving the efficiency of debt management and financial recovery for borrowers. The platform provides a smart and borrower-friendly solution for analyzing, negotiating, and resolving loan obligations through an interactive and professional digital environment.

The project follows a modular full-stack architecture using React.js for the frontend and FastAPI with Python for backend processing, ensuring smooth communication between borrowers and system components. The structured design enables seamless integration of modules such as financial health calculation, settlement prediction, AI negotiation strategy generation, negotiation email creation, authentication management, and loan priority analysis.

By integrating AI-powered negotiation strategy generation mechanisms, the system enhances borrower experience by providing dynamic and personalized settlement recommendations based on the borrower's financial profile, loan details, overdue period, and debt stress level. The intelligent negotiation engine improves debt resolution outcomes and increases borrower confidence within the platform.

The system also incorporates structured input validation, efficient API communication, robust rule-based fallback handling, and error management mechanisms to ensure reliable and stable performance. The inclusion of JWT-based authentication, Werkzeug password hashing, SQLite persistent storage, and modular architecture improves scalability, maintainability, and overall system efficiency.

Overall, the FinRelief AI – AI-Powered Debt Relief & Financial Recovery Platform simplifies the process of debt negotiation and financial recovery, reduces manual effort in settlement letter preparation, improves borrower empowerment through AI-driven insights, and provides an efficient and scalable solution for modern AI-driven financial relief systems. The project highlights the potential of cloud-based large language models and intelligent financial algorithms in enhancing digital debt management platforms.